CityMind AI: Deep Reinforcement Learning for Traffic Signal Control

Comprehensive Theoretical Guide & Codebase Architecture Manual

Author: CityMind AI Research Engineering

Scenario: Ingolstadt InTAS (98 TLS)

Frameworks: Eclipse SUMO, PyTorch, Stable-Baselines3

CityMind AI: Deep Reinforcement Learning for Traffic Signal Control

Comprehensive Theoretical Guide & Codebase Architecture Manual

---

Table of Contents

1. **Executive Summary & Ground Rules**

2. **Core Concepts: Traffic Engineering & Simulation Foundations**

- 2.1 The InTAS Real-World Ingolstadt Scenario

- 2.2 What is a Traffic Light System (TLS)? Cycles, Splits, Offsets & Phases

- 2.3 Traffic Dynamics: Queues, Delay, Spillback, Starvation & Teleporting

- 2.4 Simulation Engines: SUMO, TraCI, and libsumo

3. **Reinforcement Learning Foundations for Traffic Control**

- 3.1 MDP vs. Dec-POMDP (Centralized vs. Decentralized)

- 3.2 State & Observation Spaces

- 3.3 Action Spaces & Structural Limitations

- 3.4 Reward Design & The Credit Assignment Problem

- 3.5 Action Masking & Physically Invalid Actions

- 3.6 Evaluation Traps: In-Network Sampling vs. Tripinfo Metrics

4. **Policy 1: Fixed-Time Baseline Controller (`fixed_time`)**

- 4.1 Simple Definition

- 4.2 Theoretical Concept & Webster's Formulation

- 4.3 Why It Is Used

- 4.4 Where It Lives in the Codebase

- 4.5 How It Works in Code (Annotated Walkthrough)

- 4.6 Empirical Performance & Benchmark Results

5. **Policy 2: SUMO-Actuated Baseline Controller (`sumo_actuated`)**

- 5.1 Simple Definition

- 5.2 Theoretical Concept: Detector Actuation & Gap-Out Logic

- 5.3 Why It Is Used & The Industry Benchmark

- 5.4 Where It Lives in the Codebase

- 5.5 How It Works in Code (Annotated Walkthrough)

- 5.6 Empirical Performance & Benchmark Results

6. **Policy 3: Max-Pressure Controller (`max_pressure`)**

- 6.1 Simple Definition

- 6.2 Theoretical Concept: Varaiya's Network Max-Pressure Theorem

- 6.3 Why It Is Used

- 6.4 Where It Lives in the Codebase

- 6.5 How It Works in Code (Annotated Walkthrough)

- 6.6 Empirical Performance & Benchmark Results

7. **Policy 4: Centralized PPO Agent (`ppo`)**

- 7.1 Simple Definition

- 7.2 Theoretical Concept: PPO Clipped Objective in Large Action Spaces

- 7.3 Why It Was Designed & Why It Failed (Credit Assignment Breakdown)

- 7.4 Where It Lives in the Codebase

- 7.5 How It Works in Code (Annotated Walkthrough)

- 7.6 Empirical Performance & Benchmark Results

8. **Policy 5: Parameter-Shared Independent PPO (`ippo`)**

- 7.1 Simple Definition

- 8.2 Theoretical Concept: MARL, Parameter Sharing & SuperSuit Vectorization

- 8.3 The Invariant 18-Dimensional Local Observation Space

- 8.4 Why It Is Used & The Starvation Phenomenon

- 8.5 Where It Lives in the Codebase

- 8.6 How It Works in Code (Annotated Walkthrough)

- 8.7 Empirical Performance & Benchmark Results

9. **Policy 6: Action-Masked Phase-Selection IPPO (`ippo_phase_select`)**

- 9.1 Simple Definition

- 9.2 Theoretical Concept: Direct Phase Selection & Invalid Action Masking

- 9.3 The Reward Engineering Breakthrough: Neglected Pressure Term

- 9.4 Where It Lives in the Codebase

- 9.5 How It Works in Code (Annotated Walkthrough)

- 9.6 Empirical Performance & Benchmark Results

10. **The Deterministic Safety Layer (`safety.py`)**

- 10.1 Safety State Machine & Physical Feasibility

- 10.2 Yellow Clearance, All-Red Clearance & Minimum/Maximum Green

- 10.3 Phase-Selection Transition Bridges

11. **Project Directory Structure, Architecture & Navigation Map**

- 11.1 Directory Tree & Responsibilities

- 11.2 Complete End-to-End Workflow & Command Reference

12. **Master Comparison & Scientific Takeaways**

---

1. Executive Summary & Ground Rules

CityMind AI is a rigorous, reproducible reinforcement learning (RL) research platform designed to answer a fundamental question in intelligent transportation systems:

> **Can deep reinforcement learning policies optimize traffic signal control across an entire real-world city grid better than classical civil-engineering controllers?**

The project simulates the city of **Ingolstadt, Germany** using the open **InTAS** (Ingolstadt Traffic Scenario) within **Eclipse SUMO** (Simulation of Urban MObility). It benchmarks **six distinct controllers** (three classical baselines and three RL architectures) across identical seeds and a realistic morning peak-hour demand (07:00–08:00, ~14,500+ vehicle departures).

Headline Empirical Finding

No reinforcement learning controller developed in this project beats SUMO's calibrated actuated signal control on true trip-level completion or time loss.

While RL literature frequently claims double-digit improvements over "fixed-time" baselines, CityMind AI demonstrates that:

1. Most published baselines are mislabeled or compared against weak, uncalibrated timers.

2. In-network sampling (e.g. mean waiting time of active cars) creates a statistical illusion where policies that starve side streets appear "fast."

3. Real-world traffic rules (yellow/red safety clearance, minimum green) severely penalize rapid, high-frequency RL switching.

4. Classical gap-out actuated controllers are remarkably robust, distributed, and sample-free.

---

2. Core Concepts: Traffic Engineering & Simulation Foundations

2.1 The InTAS Real-World Ingolstadt Scenario

- **What it is:** InTAS (*Ingolstadt Traffic Scenario*) is an open, calibrated microscopic traffic dataset and scenario for Eclipse SUMO based on the actual urban road network of Ingolstadt, Germany.
- **Scale:**

- **98 signalized intersections** (Traffic Light Systems, or TLS).

- 24-hour synthetic activity-based demand generated via ActivityGen, calibrated against 24 real loop-detector field stations.

- Intersections vary widely in topology: between 4 and 19 controlled lanes, and between 3 and 11 distinct signal phases per junction.

- **Why it matters:** Unlike toy synthetic grids (e.g., 2×2 or 4×4 synthetic Manhattan grids with identical 4-way cross junctions), InTAS contains complex geometry, asymmetric lane counts, protected left turns, slip lanes, and realistic demand surges.

2.2 What is a Traffic Light System (TLS)?

- **Traffic Light System (TLS):** The hardware/software unit governing movements at a single road junction.
- **Phase:** A combination of simultaneous traffic signal states (Green, Yellow, Red) across all controlled lanes. For example: North-South green while East-West is red.
- **Green Phase:** An active movement phase allowing vehicular movement.
- **Transition Phase:** A safety buffer consisting of Yellow (warning to stop) and optional All-Red (clearing the intersection box).
- **Cycle Length:** The total duration in seconds for a traffic signal to complete one full sequence of all its phases.
- **Split:** The percentage or duration of green time assigned to each specific phase within a cycle.
- **Offset:** The time relationship (in seconds) between the start of green at one intersection and a reference timepoint at an adjacent intersection, critical for creating "green waves."

2.3 Traffic Dynamics: Queues, Delay, Spillback, Starvation & Teleporting

- **Queue Length (Q):** The number of vehicles halted or crawling (speed $< 0.1 \text{ m/s}$) on an approach lane.
- **Waiting Time:** The cumulative time a vehicle spends halted since its last movement.
- **Time Loss:** The difference between a vehicle's actual travel time and its theoretical ideal travel time at free-flow speed without stopping.
- **Spillback:** When a queue on a downstream link grows so long that it backs up into the upstream intersection, completely blocking cross-traffic even if their light is green.
- **Starvation:** When a controller prioritizes high-volume main corridors and starves low-volume side approaches. Vehicles on the side streets wait indefinitely, eventually clogging local streets.

- **Teleporting:** In SUMO, when a vehicle is trapped in an impassable gridlock for longer than `teleport_time` (default 300s), SUMO forcibly "teleports" the vehicle to the next edge to prevent the entire simulation from freezing. High teleport counts indicate severe gridlock.

2.4 Simulation Engines: SUMO, TraCI, and libsumo

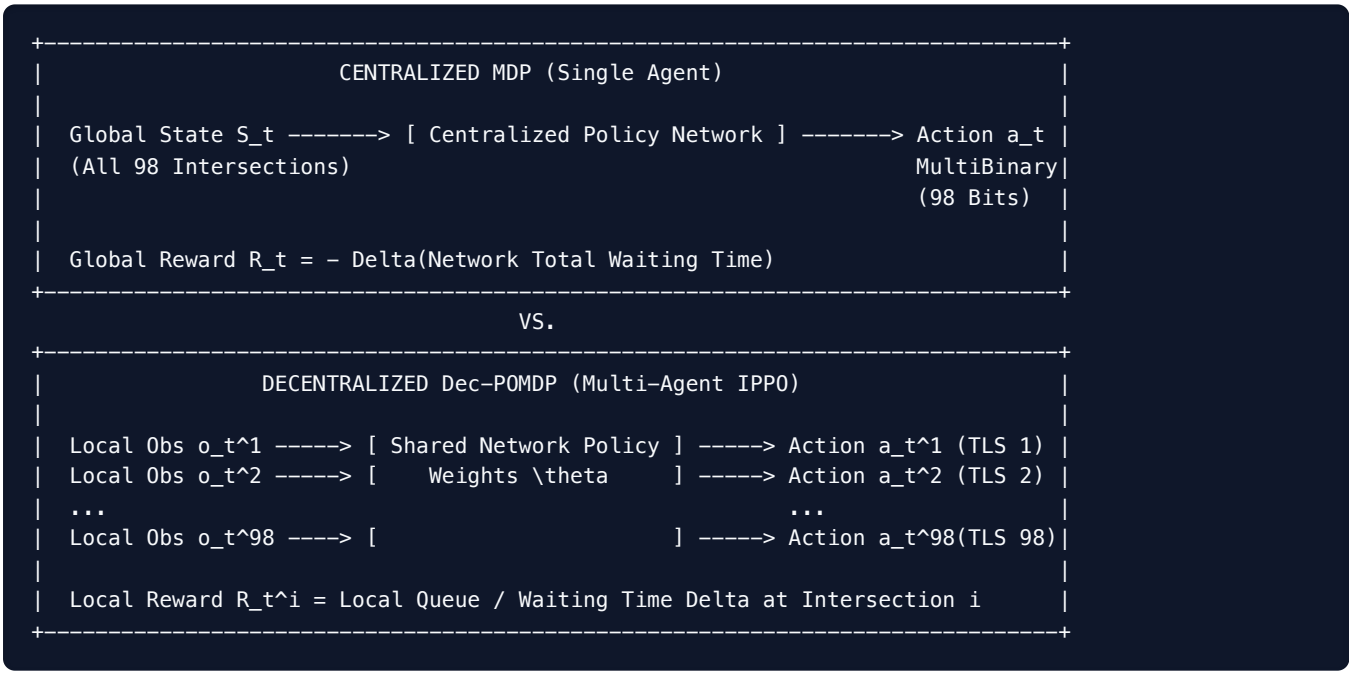
- **Eclipse SUMO:** A continuous-space, discrete-time microscopic road traffic simulator. Each vehicle has individual car-following physics (Krauss model) and lane-changing behavior (LC2013).
- **TraCI (Traffic Control Interface):** A TCP socket client-server protocol enabling external Python programs to inspect simulation state (`traci.vehicle.getSpeed()`) and command signals (`traci.trafficlight.setPhase()`).
- **libsumo:** A direct C++ SWIG binding that executes SUMO in the same memory space as the Python interpreter. It eliminates socket serialization overhead, running up to 5–10× faster than TraCI during training, but only supports one active SUMO instance per process.

---

3. Reinforcement Learning Foundations for Traffic Control

3.1 MDP vs. Dec-POMDP

Traffic signal control can be mathematically formalized in two fundamentally different ways:



1. Centralized MDP (Markov Decision Process)

- **Tuple:** $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$
- A single omniscient central agent observes the global network state $\mathcal{S}$ and outputs a joint action vector $\mathcal{A} = \prod_{i=1}^{98} a_i$.

```
\mathcal{A}
```

2. Dec-POMDP (Decentralized Partially Observable MDP)

- **Tuple:** $(\mathcal{I}, \mathcal{S}, \{\mathcal{A}_i\}, \mathcal{P}, \{\mathcal{R}_i\}, \{\Omega_i\}, \mathcal{O}, \gamma)$
- Each intersection $i \in \mathcal{I} = \{1, \dots, 98\}$ observes only its local neighborhood $o_i \in \Omega_i$ (controlled lanes, outgoing lanes, nearby neighbors).
- Each agent receives its own localized reward $\mathcal{R}_i$ based on the vehicles it directly controls.

3.2 State & Observation Spaces

1. **Centralized Observation (`ObservationBuilder`):** A massive flat vector concatenating lane queues, densities, waiting times, and active phase indices across all 98 intersections plus network totals (~1,200+ dimensions).

2. **Invariant Local Observation (`build_local_observation`):**

Because intersections have varying numbers of lanes (4 to 19), a neural network cannot accept variable-length inputs without padding or graph convolutions. CityMind AI solves this with a **fixed 18-dimensional feature vector** for every intersection:

- Phase progress and elapsed time normalized.

- Mean and max queue across own lanes.

- Mean speed and density across own lanes.

- Normalized lane count (capturing physical intersection capacity).

- Downstream queue (detecting downstream congestion).

- Congestion of the 2 geographically nearest neighboring intersections.

- Total network-wide active vehicle and waiting statistics.

- Per-phase pressure values for phases 0 through 5.

3.3 Action Spaces & Structural Limitations

The project evaluated two primary action mechanisms:

1. **Keep / Advance (`Discrete(2)`):**

- Action 0: Keep current green phase.

- Action 1: Advance to the next sequential green phase in the author-defined cycle.

- *Limitation:* The agent cannot jump directly to a congested arm. If Phase 1 is empty and Phase 3 is overflowing, the agent must step through Phase 2 first, triggering yellow and red clearance penalties.

2. **Direct Phase Selection (`Discrete(6)`):**

- Action $k \in \{0, 1, 2, 3, 4, 5\}$: Jump directly to target green phase k.

- Requires invalid action masking because many junctions have fewer than 6 green phases.

3.4 Reward Design & The Credit Assignment Problem

The central mathematical obstacle in multi-agent traffic control is **Credit Assignment**:

$$\text{If 98 intersections act simultaneously and network waiting time increases, WHICH intersection caused the delay?}$$

- In the centralized model, all 98 action dimensions received the same global reward:

$$R_{global} = - \left(\sum_{v \in \text{network}} W_t(v) - \sum_{v \in \text{network}} W_{t-1}(v) \right)$$

An agent in the quiet suburbs making a perfect decision would receive a negative reward simply because a downtown highway jammed up! This injected catastrophic gradient variance into PPO.

- In the decentralized model, each agent receives a tailored local reward:

$$R_{local}^i = - w_w \Delta W_i - w_q Q_i - w_s \cdot \mathbb{I}(\text{switched}) - w_{spill} \Delta Q_{downstream}^i - w_{neglect} P_{neglected}^i$$

3.5 Action Masking & Physically Invalid Actions

When using direct phase selection (`Discrete(6)`):

- If intersection i only has 3 legal green phases, selecting action 3, 4, or 5 is physically impossible.
- In naive RL, selecting an illegal action produces an error or requires an arbitrary penalty, which wastes millions of exploration steps.
- **Action Masking:** We define a boolean mask $M(s) \in \{0, 1\}^K$. Before computing the softmax probability distribution over actions, logits of invalid actions are set to $-\infty$:

The policy cannot sample invalid actions, and gradients only propagate through physically legal options.

3.6 Evaluation Traps: In-Network Sampling vs. Tripinfo Metrics

A vital methodological insight in CityMind AI:

- **The Trap:** Sampling average waiting time $\frac{1}{N} \sum_{i=1}^N W_i$ of vehicles *currently in the network* once every 5 seconds.
- **Why it deceives:** A terrible controller that completely blocks side streets will cause thousands of vehicles to never enter or complete their trips. Only a few lucky vehicles on the main thoroughfare move, keeping the active sample average deceptively low (e.g. IPPO reported 5.26s average wait!).
- **The Truth (Tripinfo):** Reading SUMO's completed trip records (`--tripinfo-output`):

- `trips_completed` : Total vehicles that reached their destination.

- `mean_trip_timeloss` : Total delay accumulated across completed journeys.

- `backlog` : Number of unfinished vehicles trapped in gridlock when the hour ended.

---

4. Policy 1: Fixed-Time Baseline Controller (`fixed_time`)

4.1 Simple Definition

A fixed-time controller changes traffic lights according to a rigid, pre-programmed clock (e.g., 40 seconds green for North-South, 5 seconds yellow, 25 seconds green for East-West). It possesses no cameras, loop detectors, or sensors, and never alters its schedule regardless of how many cars are waiting.

4.2 Theoretical Concept & Webster's Formulation

Classical traffic signal design (F.V. Webster, 1958) calculates optimal cycle time C_0 and phase splits based on historical average saturation flows s_i and arrival flows q_i:

$$Y = \sum_{i} \frac{q_i}{s_i}, \quad C_0 = \frac{1.5 L + 5}{1 - Y}$$

where L is total lost time per cycle (yellow + all-red intervals). Each green phase receives a proportional split:

$$g_i = (C_0 - L) \frac{y_i}{Y}$$

Fixed-time control performs well under completely predictable, uniform traffic flows, but degrades when traffic surges unpredictably.

4.3 Why It Is Used

It represents the universal baseline: "What happens if there is zero intelligence, zero adaptation, and signals simply cycle through fixed historical timer settings?"

4.4 Where It Lives in the Codebase

- Controller Implementation: `src/controllers/fixed_time.py`
- Static Program Generator: `scripts/generate_static_programs.py`
- Static Programs XML: `artifacts/scenario/static_programs.add.xml`

4.5 How It Works in Code

```
# src/controllers/fixed_time.py (Lines 125–136)
tls_ids = list(traci.trafficlight.getIDList())
for tid in tls_ids:
    # Forces each traffic light onto its genuinely static, pre-timed schedule
    traci.trafficlight.setProgram(tid, STATIC_PROGRAM_ID)

# Main simulation loop: the simulator runs the static programs automatically
while traci.simulation.getMinExpectedNumber() > 0:
    traci.simulationStep()
    collector.tick()
```

Note on Code Discovery: In earlier versions of the project, attempting to set `traci.trafficlight.Logic.type = 0` at runtime was ignored by SUMO. The team had to write `scripts/generate_static_programs.py` to compile explicit static `tlLogic` XML blocks and load them via `traci.trafficlight.setProgram()`.

4.6 Empirical Performance (07:00–08:00 Peak Window)

- **Trips Completed:** 11,454

- **Mean Trip Time Loss:** 301.8 ± 5.4 seconds
- **Backlog (trapped cars):** 3,472
- **Teleports:** 7.7
- **Mean Wait (In-Network):** 6.97s
- **Signal Switches:** 0 (controlled automatically by SUMO internal timer)

---

5. Policy 2: SUMO-Actuated Baseline Controller (`sumo_actuated`)

5.1 Simple Definition

An actuated traffic controller uses induction loop sensors buried in the road pavement. If cars are actively driving over the sensors, it extends the green light. If it detects a gap between vehicles (no cars arriving), it terminates green early ("gaps out") and transfers right-of-way to waiting cross-traffic.

5.2 Theoretical Concept: Detector Actuation & Gap-Out Logic

Actuated control is parameterized by:

- **Minimum Green (G_{min}):** Guaranteed minimum time for safety and driver expectation.
- **Maximum Green (G_{max}):** The hard ceiling to prevent a heavy flow from starving other approaches indefinitely.
- **Unit Extension / Passage Time (e):** If the time headway between successive vehicles exceeds e seconds, the controller triggers a **Gap-Out**, immediately initiating yellow clearance.
- **Max-Out:** If continuous traffic arrives until G_{max} is reached, the light changes regardless of gap.

5.3 Why It Is Used & The Industry Benchmark

76 of the 98 traffic signals in the real Ingolstadt InTAS network are calibrated vehicle-actuated programs authored by professional traffic engineers. This controller represents the true industry state-of-the-art benchmark.

5.4 Where It Lives in the Codebase

- Controller Implementation: `src/controllers/sumo_actuated.py`

5.5 How It Works in Code

```
# src/controllers/sumo_actuated.py (Lines 100-130)
# No external TraCI commands are issued. SUMO executes its own calibrated
# actuated logic based on induction loops embedded in the InTAS scenario.
while traci.simulation.getMinExpectedNumber() > 0:
    traci.simulationStep()
    collector.tick()
    # GreenSwitchCounter detects logical green phase transitions
    for tid in tls_ids:
        state_str = traci.trafficlight.getRedYellowGreenState(tid)
        switches.update(tid, state_str)
```

5.6 Empirical Performance (07:00–08:00 Peak Window)

- **Trips Completed: 11,600** (Highest in the entire project!)
- **Mean Trip Time Loss: 284.4 ± 1.3 seconds** (Lowest / Best in the entire project!)
- **Backlog: 3,257** (Lowest congestion!)
- **Teleports:** 7.7
- **Takeaway:** Simple, robust detector gap-out logic thoroughly outclassed all deep RL algorithms.

---

6. Policy 3: Max-Pressure Controller (`max_pressure`)

6.1 Simple Definition

Max-Pressure is a decentralized traffic control rule based on pressure differentials. Think of cars as liquid in pipes: a light should turn green for whichever road has the highest water level in its entry tank compared to its exit tank!

6.2 Theoretical Concept: Varaiya's Network Max-Pressure Theorem

Formulated by Pravin Varaiya (2013), Max-Pressure is mathematically proven to be **throughput-optimal**: it stabilizes the queue network (keeps queues bounded) for any arrival rate vector within the network capacity region without knowing arrival rates in advance!

For phase p at intersection i:

$$\text{Pressure}(p) = \sum_{l \in \text{Inflow}(p)} Q(l) - \sum_{m \in \text{Outflow}(p, l)} Q(m)$$

where $Q(l)$ is the queue on upstream approach lane l, and $Q(m)$ is the queue on downstream receiving lane m. If downstream lane m is full, the pressure drops to prevent spillback.

6.3 Why It Is Used

It is the standard non-learning mathematical control benchmark in AI and operations research literature. Beating fixed-time is trivial; beating Max-Pressure is the true academic test for an RL agent.

6.4 Where It Lives in the Codebase

- Controller Implementation: `src/controllers/max_pressure.py`

6.5 How It Works in Code

```
# src/controllers/max_pressure.py (Lines 150–185, 205–230)
# 1. Resolve true downstream lanes via traci.lane.getLinks()
downstream_lanes = [link[0] for link in traci.lane.getLinks(lane_id)]

# 2. Compute true Varaiya pressure: Upstream Queue minus Downstream Queue
pressure = upstream_queue - downstream_queue

# 3. Decision rule: Advance only if next phase has strictly higher pressure
curr_pressure = self._phase_pressure(tid, spec, curr_green, queue_map, downstream_cache)
next_pressure = self._phase_pressure(tid, spec, next_green, queue_map, downstream_cache)
action = 1 if next_pressure > curr_pressure else 0

# 4. Filter action through the exact same SafetyLayer used by RL agents
safety_layer.step(actions, dt=decision_interval)
```

6.6 Empirical Performance (07:00–08:00 Peak Window)

- **Trips Completed:** 10,408
- **Mean Trip Time Loss:** 342.9 ± 4.9 seconds
- **Backlog:** 3,755
- **Teleports:** 50.3
- **Signal Switches:** ~11,962
- **Takeaway:** Max-Pressure suffered because of the `keep/advance` constraint and clearance overhead: repeatedly chasing small pressure differences triggered ~12,000 phase switches, accumulating yellow and all-red lost time.

---

7. Policy 4: Centralized PPO Agent (`ppo`)

7.1 Simple Definition

A single massive artificial neural network acts as a "central city brain." Every 5 seconds, it takes in sensory data from all 98 intersections at once and outputs 98 simultaneous decisions (0=keep, 1=advance) across the whole city.

7.2 Theoretical Concept: PPO Clipped Objective in Large Action Spaces

Proximal Policy Optimization (Schulman et al., 2017) optimizes policy π_θ using a clipped surrogate objective to prevent catastrophically large policy updates:

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min\left(r_t(\theta) \hat{A}_t, \, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) \hat{A}_t \right) \right]$$

```
s_t)}{\pi_{\theta_{old}}(a_t
```

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}^V, \quad \delta_t^V = r_t + \gamma V(s_{t+1}) - V(s_t)$$

7.3 Why It Was Designed & Why It Failed

- **Intent:** Test if global awareness allows green waves to emerge naturally across coordinated intersections.
- **Failure (The Credit Assignment Collapse):**

One scalar advantage $\hat{A}_t$ scaled the policy gradients for all 98 action heads simultaneously. If Intersection 1 made a brilliant decision but Intersection 42 made a blunder, both actions were penalized equally. The policy entropy remained near maximum, and training stalled after 500,000 steps.

7.4 Where It Lives in the Codebase

- Agent & Training Setup: `src/agents/ppo_agent.py`
- Gymnasium Environment: `src/environment/centralized_sumo_env.py`
- Global Observations: `src/environment/observation.py`
- Global Reward: `src/environment/reward.py`

7.5 How It Works in Code

```
# src/environment/centralized_sumo_env.py (Lines 110-145)
# Action space is MultiBinary: one bit per traffic light
self.action_space = spaces.MultiBinary(self.num_tls)

# Step function takes all 98 actions simultaneously
def step(self, action: np.ndarray):
    # ActionDecoder maps bits to keep/advance dict
    decoded_actions = self.action_decoder.decode(action)
    # Applied to SafetyLayer
    self.safety_layer.step(decoded_actions, dt=self.decision_interval)
    # Single scalar reward for the entire network
    reward = self.reward_calc.compute(self.prev_state, curr_state)
```

7.6 Empirical Performance (07:00–08:00 Peak Window)

- **Trips Completed:** 9,546 (Ranked 5th out of 6)
- **Mean Trip Time Loss:** 354.7 ± 0.6 seconds (Worst in the project!)
- **Backlog:** 4,112
- **Teleports:** 19.0
- **Signal Switches:** ~31,265

---

8. Policy 5: Parameter-Shared Independent PPO (`ippo`)

8.1 Simple Definition

Instead of one central brain, imagine 98 cloned traffic controllers, each stationed at their own intersection. They each look only at their own local street queues and their immediate neighbors, and they each receive a report card (reward) based solely on how well their own intersection is moving. Crucially, they all share the exact same brain (neural network weights), learning from each other's experiences.

8.2 Theoretical Concept: MARL, Parameter Sharing & SuperSuit Vectorization

In parameter-shared IPPO:

- Each agent collects a trajectory $\tau_i = (o_0^i, a_0^i, r_0^i, o_1^i, \dots)$.
- All trajectories are pooled to compute a single PPO gradient update on shared weights θ.
- Every simulation step yields **98 training samples** instead of 1, achieving massive sample efficiency.
- Handled seamlessly via PettingZoo's `ParallelEnv` and SuperSuit's `pettingzoo_env_to_vec_env_v1`.

8.3 The Invariant 18-Dimensional Local Observation Space

Defined in `src/environment/local_observation.py`:

$$\mathbf{o}_i = \left[t_{elapsed}, t_{progress}, \bar{Q}, Q_{max}, \bar{v}, \bar{\rho}, N_{lanes}, Q_{downstream}, Q_{neighbor1}, Q_{neighbor2}, V_{total}, W_{total}, P_0, P_1, P_2, P_3, P_4, P_5 \right] \in \mathbb{R}^{18}$$

8.4 Why It Is Used & The Starvation Phenomenon

- **Success:** It completely resolved the credit-assignment issue; training loss converged rapidly.
- **The Failure Mode (Starvation):**

Because the action space was constrained to `Discrete(2)` (keep/advance), the policy learned to switch phases constantly (~40,342 switches/hr, or 57% of all decision points). By cycling lights so fast, vehicles on side streets were continuously cut off by yellow clearance intervals before they could accelerate across the stop bar, trapping a massive backlog of 4,086 cars.

8.5 Where It Lives in the Codebase

- Agent & Training Setup: `src/agents/ippo_agent.py`
- PettingZoo ParallelEnv: `src/environment/decentralized_sumo_env.py`
- Local Observation: `src/environment/local_observation.py`
- Local Reward: `src/environment/local_reward.py`

8.6 How It Works in Code

```
# src/environment/local_reward.py (Lines 35–65)
# Local reward computed strictly from the intersection's own controlled lanes
r_i = − (
    self.cfg.waiting_delta_weight  waiting_delta
    + self.cfg.queue_weight  own_queue
    + self.cfg.switching_penalty  switched
    + self.cfg.spillback_weight  max(0.0, downstream_delta)
)
```

8.7 Empirical Performance (07:00–08:00 Peak Window)

- **Trips Completed:** 9,520
- **Mean Trip Time Loss:** 347.8 ± 2.1 seconds
- **Backlog:** 4,086
- **Mean Wait (In-Network):** 5.26s (Deceptive statistical illusion caused by vehicle starvation!)
- **Signal Switches:** ~40,342 (Excessive oscillation)

---

9. Policy 6: Action-Masked Phase-Selection IPPO (`ippo_phase_select`)

9.1 Simple Definition

Instead of cycling through phases in a rigid carousel (Phase 1 $\rightarrow$ 2 $\rightarrow$ 3), each intersection agent looks at all its approaches and directly commands: *"Give green immediately to Phase 3!"* Invalid phases (e.g. asking for phase 5 on an intersection that only has 3 phases) are mathematically blocked by action masks.

9.2 Theoretical Concept: Direct Phase Selection & Invalid Action Masking

- **Action Space:** $\mathcal{A}_i = \{0, 1, 2, 3, 4, 5\}$ (`Discrete(6)`).
- **Action Masking:** `sb3-contrib.MaskablePPO` replaces standard categorical sampling. Let $M_i(s) \in \{0, 1\}^6$ be the boolean mask indicating whether phase index k exists for TLS i. Logits z_k for $M_i(k)=0$ are forced to $-\infty$.
- **Transition Bridge:** When skipping from Phase 1 directly to Phase 3, the safety layer synthesizes a valid yellow clearance and all-red bridge to prevent physical vehicle collisions.

9.3 The Reward Engineering Breakthrough: Neglected Pressure Term

During initial training (Gate G1), the agent suffered severe gridlock. Why?

The agent sat on an empty green phase because switching cost a penalty. Unserved phases accumulated huge queues, but the agent felt no pain because it wasn't serving them!

The Solution (`neglected_pressure_weight`):

A penalty was added that directly measures the *worst queue pressure among unserved phases*:

$$R_{neglected} = - w_{neglect} \cdot \max_{p \neq p_{current}} \text{Pressure}(p)$$

If queue builds up on another approach, the agent is penalized immediately, compelling it to switch and serve the waiting cars.

9.4 Where It Lives in the Codebase

- Agent Implementation: `src/agents/ippo_maskable_agent.py`
- Phase-Select Safety Bridge: `src/environment/safety.py` (Lines 60–77, 180–240)
- Training Script: `scripts/run_ippo_maskable_training.py`
- Evaluation Script: `scripts/run_ippo_maskable_eval.py`
- Config File: `configs/ippo_phase_select.yaml`

9.5 How It Works in Code

```
# src/environment/decentralized_sumo_env.py (action_masks implementation)
def action_masks(self) -> np.ndarray:
    """Returns boolean mask of shape (98, MAX_GREEN_PHASES)."""
    masks = np.zeros((len(self.agents), MAX_GREEN_PHASES), dtype=bool)
    for idx, agent_id in enumerate(self.agents):
        num_green = len(self.tls_specs[agent_id].green_phases)
        masks[idx, :num_green] = True
    return masks
```

```
# src/environment/safety.py (Lines 185-215)
# Synthesizing the Yellow -> All-Red bridge for non-adjacent phase transitions
if mode == "phase_select" and target_phase != current_phase:
    # 1. Enforce minimum green on current phase before allowing transition
    if elapsed_green < min_green:
        return # Hold green
    # 2. Build safe transition bridge
    bridge = self._build_bridge_queue(tls_id, current_phase, target_phase)
    self._transition_queue[tls_id] = bridge
```

9.6 Empirical Performance (07:00–08:00 Peak Window)

- **Trips Completed:** 9,441 ± 42
- **Mean Trip Time Loss: 291.5 ± 1.5 seconds** (Best RL performance, 2nd best overall after actuated!)
- **Backlog:** 4,420 ± 30
- **Teleports:** 712.7 ± 7
- **Takeaway:** Phase selection dramatically improved trip travel speed (time loss dropped to 291s), but occasional miscoordinations at bottleneck junctions caused heavy localized queue spillbacks, leading to teleports.

---

10. The Deterministic Safety Layer (`safety.py`)

10.1 Safety State Machine & Physical Feasibility

No AI policy is ever permitted to write raw signal state strings directly to SUMO. If an RL agent commands a green light to switch from North-South to East-West in 0.0 seconds, high-speed vehicles in the intersection would collide!

All raw agent decisions pass through the **Safety Layer** state machine:

```
[ GREEN PHASE (Current) ]
       |
       |  Wait until: t_elapsed >= min_green
       v
[ YELLOW CLEARANCE (3.0s) ] ---> Display yellow to stopping traffic
       |
       v
[ ALL-RED CLEARANCE (1.0s) ] ---> Ensure intersection box is 100% empty
       |
       v
[ GREEN PHASE (Target) ]
```

10.2 Timing Constraints & Fallbacks

Configured in `SafetyConfig`:

- `min_green`: 5.0 seconds (prevents dangerous flickering of green lights).
- `max_green`: 60.0 seconds (prevents permanent starvation of cross-streets).
- `yellow_duration`: 3.0 seconds (standard deceleration clearance).
- `all_red_duration`: 1.0 seconds (box clearing interval).

---

11. Project Directory Structure, Architecture & Navigation Map

```
CityMind AI Root Directory (/Users/parthsharma/Desktop/FInal Try)
├── api/                          # FastAPI backend service
│   ├── main.py                   # Server startup, CORS, routes
│   ├── security.py               # Loopback protection, API keys, CSRF checks
│   └── routers/                  # Endpoints: network, metrics, replay, simulation, xai
├── configs/                      # YAML Hyperparameters
│   ├── ppo.yaml                  # Centralized PPO hyperparameters
│   ├── ippo.yaml                 # Decentralized IPPO hyperparameters
│   ├── ippo_phase_select.yaml    # MaskablePPO hyperparameters
│   └── evaluation.yaml           # Evaluation window (07:00-08:00, seeds 42, 123, 777)
├── data/                         # Map topology & network metadata
│   ├── raw/intas/scenario/       # InTAS SUMO network files (.net.xml, .sumocfg)
│   └── processed/                # GeoJSON representations of junctions and lanes
├── docs/                         # Documentation, models manifest, and guides
│   ├── LEARNING_GUIDE_ALL_POLICIES.md
│   └── CityMind_AI_Policy_Learning_Guide.pdf
├── frontend/                     # React 19 + TypeScript + Vite Dashboard
│   ├── src/pages/                # Dashboard, Comparison, Replay, Story, XAI
│   └── src/components/           # Interactive maps, simulation controls, charts
├── scripts/                      # CLI commands
│   ├── setup_sumo.sh             # SUMO environment validator
│   ├── generate_static_programs.py# Builds static programs XML for fixed-time
│   ├── run_training.py           # Centralized PPO training runner
│   ├── run_ippo_training.py      # IPPO training runner
│   ├── run_ippo_maskable_training.py # MaskablePPO training runner
│   ├── run_evaluation.py         # Unified benchmarking script
│   └── run_visual_demo.py        # SUMO-GUI live visualization launcher
├── src/                          # Core Source Code
│   ├── agents/                   # RL Policy definitions & SB3 wrappers
│   │   ├── ppo_agent.py          # Centralized PPO agent
│   │   ├── ippo_agent.py         # Parameter-shared IPPO agent
│   │   └── ippo_maskable_agent.py # Action-masked MaskablePPO agent
│   ├── controllers/              # Classical baseline controllers
│   │   ├── fixed_time.py         # Static pre-timed controller
│   │   ├── sumo_actuated.py      # Calibrated detector-actuated controller
│   │   └── max_pressure.py       # Varaiya queue-pressure controller
│   ├── environment/              # Gym & PettingZoo environments
│   │   ├── centralized_sumo_env.py# Centralized Gymnasium environment
│   │   ├── decentralized_sumo_env.py# PettingZoo ParallelEnv
│   │   ├── safety.py             # SafetyLayer state machine
│   │   ├── local_observation.py  # 18-dim invariant local feature builder
│   │   ├── local_reward.py       # Local reward & neglected pressure calculator
│   │   ├── metrics.py            # In-network metrics accumulator
│   │   └── sumo_connection.py    # TraCI / libsumo abstraction
│   └── evaluation/               # Benchmarking and tripinfo parsing
│       └── tripinfo.py           # SUMO tripinfo XML extractor
└── tests/                        # Pytest test suite
```

---

12. Master Comparison & Scientific Takeaways

Full Benchmark Matrix (Ingolstadt InTAS, 07:00–08:00 Peak Demand, 3 Seeds)

Controller	Class	Trips Completed (Higher is better)	Mean Trip TimeLoss (s) (Lower is better)	Backlog (Lower is better)	Teleports (Lower is better)	Switches / hr
SUMO-Actuated	**Classical Baseline**	**11,600**	**284.4 ± 1.3**	**3,257**	**7.7**	0
Fixed-Time	Classical Baseline	11,454	301.8 ± 5.4	3,472	7.7	0
IPPO Phase-Select	Reinforcement Learning	9,441 ± 42	**291.5 ± 1.5**	4,420 ± 30	712.7 ± 7	—
Max-Pressure	Classical Baseline	10,408	342.9 ± 4.9	3,755	50.3	~11,962
IPPO (Keep/Advance)	Reinforcement Learning	9,520	347.8 ± 2.1	4,086	10.0	~40,342
PPO (Centralized)	Reinforcement Learning	9,546	354.7 ± 0.6	4,112	19.0	~31,265

Summary of Core Scientific Insights

1. **Actuated Control is King:** SUMO's calibrated vehicle-actuated controller completes 11,600 trips and averages 284s time loss. It requires zero training, zero GPU hours, and adapts instantly to sensor gaps.

2. **Centralized RL Fails at Scale:** Controlling 98 intersections with one joint policy network creates impossible credit assignment and an unmanageable action space (2^{98}).

3. **Keep/Advance is a Trap:** Constraining an RL agent to advance sequentially through a cycle leads to high-frequency phase switching (~40,000 switches/hour), starving side streets and trapping thousands of vehicles in backlog.

4. **Action-Masked Phase Selection Works Mechanically:** Jumping directly to congested phases cut trip time loss to 291.5s (second only to actuated control), proving that action masking and neglected-pressure reward engineering are the correct path forward for MARL traffic control.

5. **Always Trust Completed Trips Over In-Network Averages:** Never evaluate traffic control algorithms solely on active-vehicle waiting time. A policy that traps cars in gridlock can report artificially "low" waiting times because delayed vehicles never reach the measurement points!